Snake: Homework vs. Class Version

A side-by-side review of techniques, strengths, weaknesses and bugs. Generated 10 November 2026.

Yours: `js/73/snakeGame.js` (516 lines) + `snakeGame.html`, `snakeGame.css`
Teacher's: `class/js/74/snake.js` (212 lines) + `snake.html`

Tags: YOURS BETTER TEACHER'S BETTER TRADE-OFF BOTH NEGLECT BUG. Highlighted lines in code: yellow = the line being discussed, red = a defect.

At a glance

TOPIC	YOURS	TEACHER'S	VERDICT
Data model	Occupancy grid (`squares[]` with status) + separate head, body-part class, tail object, and a position history	One `Snake` class holding an array of `{x, y}` segments	TEACHER
Movement	Push every head position into `snakeLocations`; parts copy from history	Pop the tail, move it to the new head spot, `unshift` it	TEACHER
Self-collision	O(1) lookup in the grid	Loop over segments	TRADE-OFF
Game loop	`setInterval`, cleared and restarted on speed-up	Recursive `setTimeout` that reads `speed` each tick	TEACHER
Rendering	Clears single cells; creates a new `<img>` for every piece every tick	Clears the whole canvas; redraws from images loaded once	TEACHER
Graphics	Direction-aware head/tail, corner pieces, open mouth on apple	Head image + green squares	YOURS
Blocking reverse turns	None	`opposites` lookup map	TEACHER
Apple placement	Picks a random *empty grid cell*	Random pixel snapped to the grid (has an off-by-one bug)	YOURS
Sound	Music, mute/unmute, game-over sequence	Two effects; resets `currentTime`	MIXED
Game flow / UI	Start/restart button, DOM score, game-over card	Starts on load, score drawn on canvas, no restart	YOURS
Code size / DRY	516 lines; crash logic repeated 5×	212 lines	TEACHER
Window resize mid-game	Neither version adjusts the snake or apple when the canvas changes size		BOTH
Fast key presses	Neither buffers input, so two quick keys within one tick can still reverse the snake		BOTH

1. Data model: how the snake is represented

TEACHER'S BETTER

> **TECHNIQUE**
> Teacher: *one source of truth*. The snake is an ordered list of segments, where index 0 is the head. Yours: *several parallel structures* (an occupancy grid, three kinds of snake piece, and a position history) that must be kept in sync by hand.

Teacher: snake.js 23–31

```
23 class Snake {
24   segments = [{
25     x: -SNAKE_SIZE,
26     y: 0
27   }];
28
29   get head() {
30     return this.segments[0];
31   }
```

Yours: snakeGame.js 90–104, 232–238, 277–288

```
90 const snakeHead = {
91   top: 0,
92   bottom: 64,
93   left: 0,
94   right: 64,
     ...
232 const tail = {
233   top: 0,  bottom: 64,
     ...
277 class SnakePart {
278   constructor(top, bottom, left, right,
direction) {
```

Teacher's approach. Every segment is the same shape, so moving, drawing, growing and checking collisions all work on one array. It uses three modern features worth learning:

- **Public class fields** (`segments = [...]`): state declared directly in the class body, with no constructor needed.
- **Getter** (`get head()`): a computed property, so `snake.head.x` reads naturally and can never fall out of sync.
- **`Array.prototype.at(-1)`** in `grow()` to get the last element.

Your approach. The head is an object literal, the tail is another object literal, and the body is a `class`. That's three shapes doing almost the same job, which is why the file is roughly 2.5× longer. Each piece also stores `top`, `bottom`, `left` *and* `right`, but `bottom` is always `top + 64` and `right` is always `left + 64`. That redundant state has to be updated in every branch of `move()` (lines 161–162, 176–177, 191–192, 206–207). Storing just `x, y` (or even `row, col`) would remove that work.

Methods in the constructor. In `SnakePart` you assign `this.draw = function…` and `this.move = function…` inside the constructor (lines 289, 324). That works, but it creates a new copy of each function for every body part. Declaring them as class methods (`draw() { … }`), as the teacher does, puts one shared copy on the prototype.

2. Movement: making the body follow the head

TEACHER'S BETTER

> **TECHNIQUE**
> Teacher: *tail recycling* (a queue-like shift). Remove the last segment, give it the new head coordinates, and put it at the front. Only one object changes per tick. Yours: *position history*. Record every place the head has been, then send each body part to the matching earlier entry.

Teacher: snake.js 73–78

```
73 } else {
74   const oldTail = this.segments.pop();
75   oldTail.x = tempX;
76   oldTail.y = tempY;
77   this.segments.unshift(oldTail);
78 }
```

Growing: snake.js 33–38

```
33 grow() {
34   this.segments.push({
35     x: this.segments.at(-1).x,
36     y: this.segments.at(-1).y
37   });
38 }
```

Yours: snakeGame.js 227–228, 357–364

```
227 const loc = new Locations(this.top, ...);
228 snakeLocations.push(loc);

357 function moveSnakeParts() {
358   let j;
359   let i;
360   for (i = snakeParts.length - 1,
          j = snakeLocations.length - 2; i >= 0;
i--, j--) {
361     snakeParts[i].move(snakeLocations[j].top,
...
            snakeLocations[j + 1].direction);
362   }
363   tail.move(snakeLocations[j].top, ...);
364 }
```

Why the teacher's is cleaner. Each tick touches exactly one object, no matter how long the snake is. Growing is just "add a duplicate of the tail"; on the next move, the original tail stays put while the copy is the one recycled, so the snake gets one longer without any special-case code.

Credit where due. Your history approach does have an advantage: you store each location's `direction`, so a body part knows both where it came from and where it is going (`direction` vs `nextDirection`). That's what makes your corner sprites possible. The teacher's version could only work out corners by comparing neighbouring segments' coordinates.

> **Neglected: unbounded history.** `snakeLocations` gets one new entry every tick and is never trimmed. At 400 ms per tick that's about 9,000 objects an hour. That's harmless in practice, but it's a memory leak by design. You only ever need the last `snakeParts.length + 2` entries, so a `snakeLocations.shift()` once the array is longer than that would bound it.

3. Collision detection

TRADE-OFF

> TECHNIQUE
> Yours: a *spatial grid / occupancy map*. Each cell knows whether it's `'empty'`, `'snake'` or `'apple'`, so a lookup is O(1). Teacher: a *linear scan* comparing coordinates, which is O(n) but has no extra state to keep in sync.

Yours: snakeGame.js 137–145, 493–504

```
137 if (squares[this.square].status == "snake") {
138   clearInterval(gameIsOn);
     ...
144 }
145 squares[this.square].status = 'snake';

493 function findSquareNum(top, left) {
495   const columns = theCanvas.width /
SNAKE_SIZE;
498   const c = left / SNAKE_SIZE;
499   const r = top / SNAKE_SIZE;
501   const num = r * columns + c;
503   return num;
504 }
```

Teacher: snake.js 59–65

```
59 for(let i = 3; i < this.segments.length - 1;
i++) {
60   if (this.segments[i].x === tempX &&
         this.segments[i].y === tempY) {
61     gameOver = true;
62     crash.play();
63     break;
64   }
65 }
```

Your grid is a real data-structure idea. `findSquareNum` converts a 2-D position to a 1-D index (`row * columns + col`), which is row-major indexing, the same way images and matrices are stored in memory. It also makes apple placement easy (section 7). The cost is that every move has to remember to set the old cell to `'empty'` and the new one to `'snake'`, in the right order. Those writes are spread across `snakeHead.move`, `snakeHead.draw`, `tail.move`, `SnakePart.move` and `SnakePart.draw`, so they're easy to get wrong (see sections 6 and 10).

The teacher's loop bounds are deliberate. Starting at `i = 3` works because on a grid the head can't run into segments 1 or 2: those are its neck. Segment 1 is only reachable by reversing, which the `opposites` map blocks. Stopping at `length - 1` skips the tail because it moves out of the way on this same tick. Both bounds rely on facts about the game rather than brute force. The catch is that they depend on reversal being *impossible*, which it isn't fully (section 6).

Neglected in yours: separation of concerns. Your self-collision check and the game-over call live inside `snakeHead.draw()`, and `SnakePart.draw()` also writes to `squares` (line 319). Drawing code that changes game state makes bugs hard to trace. A common pattern is `update()` (all logic and state) followed by `render()` (pixels only). The teacher's version is closer to that, although it also calls `this.draw()` from inside `move()`.

4. The game loop and speeding up

TEACHER'S BETTER

> **TECHNIQUE**
> Teacher: a *self-scheduling recursive* `setTimeout`. Each tick schedules the next one using the current `speed`, so changing speed needs no extra code, and stopping is just "don't schedule again". Yours: `setInterval`, which has a fixed rate, so you have to tear it down and rebuild it to change speed.

Yours: snakeGame.js 222–224

```
222 interval = interval * .95;
223 clearInterval(gameIsOn);
224 gameIsOn = setInterval(game, interval);
```

Teacher: snake.js 127–128, 136, 151

```
127 function gameLoop() {
128   if (!gameOver) {
       ...
136       speed *= .95;
       ...
151     setTimeout(gameLoop, speed);
152   }
153 }
```

Your version works, and you correctly call `clearInterval` before every new `setInterval`, including in `playGame`, so you never end up with two loops running at once. That's a common mistake you avoided. The teacher's pattern is still simpler and also gives you a `gameOver` flag that any part of the code can set.

Startup. The teacher waits for the head image to load before starting (`snakeHead.onload = () => setTimeout(gameLoop, speed)`), so the first frame never draws a blank image. It doesn't wait for `appleImg`, though, so the apple could be missing on the first frame. Waiting for every asset (for example with `Promise.all` over each image's `load` event) would fix that.

5. Rendering

TEACHER'S TECHNIQUE YOUR VISUALS

> **TECHNIQUE**
> Teacher: *immediate-mode full redraw*. Each frame does `clearRect` on the whole canvas and draws everything again from images loaded once at startup. Yours: *dirty-rectangle* clearing (wipe only the cells that changed) plus *loading a new image element* for every sprite, every tick.

Yours: snakeGame.js 105–106, 134–136

```
105 draw: function () {
106   const snake = document.createElement('img');
     switch (this.direction) { ... snake.src = ...
}
134   snake.onload = () => {
135     context.drawImage(snake, this.left,
this.top);
136   }
```

Teacher: snake.js 129, 158–165

```
129 context.clearRect(0, 0, theCanvas.width,
                      theCanvas.height);
   ...
158 const snakeHead =
document.createElement('img');
159 snakeHead.src = 'images/snakeHead.png';
   ...
164 const appleImg =
document.createElement('img');
165 appleImg.src = 'images/apple.png';
```

Why loading images per frame is a problem. Every tick you create a new `<img>` for the head, every body part, and the tail, then draw inside `onload`. The browser cache makes this fast enough, but:

- **Drawing order isn't guaranteed.** `onload` callbacks fire whenever each image finishes, not in the order you called `draw()`.
- **Drawing is delayed until after `game()` returns,** so what you see is always slightly out of step with the game state.
- It creates garbage DOM objects every tick.

Fix: preload every sprite once into an object such as `const sprites = { headDown: img, headUp: img, … }`, then call `context.drawImage(sprites[key], x, y)` synchronously.

Why a full redraw is more robust. With dirty rectangles, anything that clears the canvas behind your back stays cleared. Resizing a canvas wipes it, and your apple is drawn *only once* (in `newApple`), so after a window resize the apple becomes invisible but is still "there". The teacher redraws everything each frame, so that kind of bug can't happen. At this size, a full redraw costs nothing.

> **Where yours clearly wins: sprite work.** Your head changes sprite with direction (`srcU/srcD/srcL/srcR`), opens its mouth when it reaches the apple (`srcOU…`, lines 111–131), the tail points the right way, and body parts pick corner sprites from `direction` + `nextDirection` (lines 292–316). That's a real "tile selection by neighbour" technique that the class version doesn't attempt. The teacher's body is plain `fillRect` squares. (A small nit there: `context.beginPath()` before `fillRect` does nothing, and `fillStyle` only needs setting once, outside the loop.)

6. Input handling

TEACHER'S BETTER BOTH NEGLECT INPUT BUFFERING

> **TECHNIQUE**
> Teacher: a *lookup table* (`opposites`) instead of branching logic, plus *switch fall-through* to treat all four arrow keys the same way.

Teacher: snake.js 167–184

```
167 const opposites = {
168   ArrowUp: 'ArrowDown',
169   ArrowDown: 'ArrowUp',
170   ArrowLeft: 'ArrowRight',
171   ArrowRight: 'ArrowLeft'
172 };
   ...
175 const isOnPiece = snake.segments.length === 1;
177 switch (e.key) {
178   case 'ArrowUp':
179   case 'ArrowDown':
180   case 'ArrowLeft':
181   case 'ArrowRight':
182     if (isOnPiece || e.key !==
opposites[direction])
183       direction = e.key;
```

Yours: snakeGame.js 445–458

```
445 window.addEventListener('keydown', e => {
446   switch (e.key) {
447     case 'ArrowUp':
448       snakeHead.direction = 'up';
449       break;
450     case 'ArrowDown':
451       snakeHead.direction = 'down';
452       break;
       ...
```

What yours neglects. There's no check for reversing. If you press the opposite arrow once the snake has a body, the head turns back into its own neck and the game ends instantly. (With only a head and tail, the two just swap places, because the tail moves out of the head's new square on the same tick.)

A lesson in the teacher's commented-out code (lines 188–211). The first attempt used `!direction === 'ArrowDown'`. Because `!` binds tighter than `===`, that becomes `(!direction) === 'ArrowDown'`, which compares a boolean to a string and is *always* false. The correct form is `direction !== 'ArrowDown'`. Replacing four near-identical branches with one table lookup is what made that bug go away.

Neglected by both: input buffering. Each key press writes `direction` immediately, but the snake only moves once per tick. Suppose you're moving Right and press Up then Left within one tick. Up passes the check, then Left passes it too, because it's checked against Up rather than the direction you last *actually moved*. The snake then reverses into itself. In the teacher's version it's worse: the self-collision loop starts at `i = 3`, so running back over segment 1 isn't even detected and the snake folds over itself. **Fix:** keep a `nextDirection` (or a small queue of key presses), check it against the direction used on the last tick, and apply it once per tick.

7. Placing the apple

YOURS BETTER TEACHER BUG

TECHNIQUE

Yours: *sample from valid cells*. Pick a random index into the grid and keep only empty cells. Teacher: *random pixel, snapped to the grid* with `x - (x % SNAKE_SIZE)`, then reject spots on the snake with `Array.prototype.some()`.

Yours: snakeGame.js 63–76

```
63 generate: function () {
64   while (!onBoard) {
65     const sq = Math.floor(Math.random() *
squares.length);
66     if (squares[sq].status == "empty") {
67       this.top = squares[sq].top;
         ...
73       onBoard = true;
```

Teacher: snake.js 106–124

```
106 while(! valid) {
107   const x =
Apple.getRandomNumber(theCanvas.width);
108   this.x = x - (x % SNAKE_SIZE);
     ...
114   if (!snake.segments.some(s => s.x === this.x
&& s.y === this.y)) {
     ...
122 static getRandomNumber(max) {
123   return Math.floor(Math.random() * (max +
1));
124 }
```

Yours is the stronger technique here. Every result is a valid cell by construction, with no snapping arithmetic involved.

Teacher bug: off-by-one. `Math.floor(Math.random() * (max + 1))` can return `max` itself. If `x === theCanvas.width` (e.g. 1280), snapping gives `1280`, which is one cell *past* the right edge. The apple is placed off-screen where it can't be reached, and the game can't progress. Each roll has about a 1 in 1,281 chance (and the same on the y axis). **Fix:** use `Math.random() * max`, or better, pick a random column and row: `Math.floor(Math.random() * cols) * SNAKE_SIZE`.

Neglected by both: if the snake ever fills the board, both `while` loops spin forever and freeze the tab. A "you win" check, or picking from a list of empty cells instead of retrying at random, would handle it.

Teacher technique worth noting: `static getRandomNumber` is a *static method*. It belongs to the class, not to an instance, so it's called as `Apple.getRandomNumber(...)`. That fits utilities that don't need `this`.

8. Sound

MIXED

Yours: snakeGame.js 212–217, 506–508

```
212 if (this.square == apple.square) {
213   onBoard = false;
214   if (!muted) {
215     chomp.play();
216     score.play();
217   }
   ...
506 crash.addEventListener('ended', () => {
507   gameOver.play();
508 });
```

Teacher: snake.js 137–138

```
137 crunch.currentTime = 0;
138 crunch.play();
```

Teacher technique: rewind before replay. Calling `play()` on audio that's already playing does nothing. Setting `currentTime = 0` first restarts it, so two quick apples both make a sound. As the snake speeds up, your `chomp` and `score` sounds will start getting skipped. Add `chomp.currentTime = 0;` (and the same for `score`).

Your techniques: chaining sounds with the `'ended'` event (crash, then the game-over jingle) is a nice event-driven touch. Background music with `loop` and `volume` and a mute toggle are also features the class version doesn't have. Starting the music from a button click also avoids the browser's autoplay block.

Markup choice: the teacher declares sounds as `<audio>` elements in HTML, while you use `new Audio(...)` in JS. Both are fine. Yours keeps all asset paths in one file.

9. Code organization

TEACHER'S BETTER

Repeated game-over logic (DRY)

This block appears five times in your file (lines 138–143, 153–158, 168–173, 183–188, 198–203):

```
clearInterval(gameIsOn);
if (!muted) { crash.play(); }
gameOverScore(currentScore);
drawSnake = false;
```

Pull it into one `function endGame() { … }`. The four wall checks in `move()` can also collapse into one: compute the new `top/left` first, then do a single bounds test, exactly like the teacher does with `tempX/tempY` (snake.js 41–57, 68–69). That "compute the candidate position, validate it, then commit it" pattern is the main structural idea to take away from the class version.

Leftovers

- An empty `regen()` method (lines 84–88) and many commented-out lines (e.g. 53, 60, 226, 274, 294, 341, 394, 434–440, 443).
- `console.log(this.direction)` and `console.log(this.nextDirection)` inside `SnakePart.move` (339–340) run for every body part on every tick, which floods the console. `console.log(squares)` also runs on every resize.
- `parseFloat(theCanvas.width)` : `canvas.width` is already a number.
- `==` is used throughout. Prefer `===`, as the teacher does, to avoid type coercion surprises.

The teacher's file has leftovers too (commented-out `x` / `y` lines and the old `switch`, plus debug `console.log`s in `Apple.move`), but far fewer.

Coupling to globals

Both versions have classes that reach out to globals. The teacher's `Snake.move` reads the global `direction` and sets `gameOver`, and `Apple.move` reads the global `snake`, so the order of `const snake = new Snake(); const apple = new Apple();` matters. Yours does this more heavily (`squares`, `currentApple`, `muted`, `gameIsOn`, …). Neither is wrong for a class exercise, but passing values in (e.g. `apple.move(snake)`) makes classes reusable and testable.

10. Bugs found

In yours

BUG	WHAT HAPPENS / WHY
BUG Double "Game Over" on wall crash	A wall crash in `snakeHead.move()` calls `gameOverScore` and `break`s *without moving* the head. `game()` then still calls `snakeHead.draw()`. The head's own cell is still marked `'snake'` from the last tick, so line 137 reads that as a self-collision and calls `gameOverScore` again. Result: two "Game Over / You scored…" blocks are added to the card. **Fix:** `return` from `game()` once the game has ended (check a `gameOver` flag), or move the self-collision check into `move()` and only run it after an actual move.
BUG Invisible apple after resize	Resizing the canvas clears it, but the apple is only drawn once, when it's created. The cell is still treated as the apple, so the player has to find it blind.
BUG Resize wipes the grid	The `resize` handler calls `fillSquaresArr()`, which resets every cell to `'empty'` mid-game. Self-collisions on old body cells are missed until those parts move again, and a new apple could be placed on the snake. `findSquareNum` also starts using the new column count, so indices shift.
BUG Reverse = instant death	No opposite-direction guard (section 6).
MINOR Missed eat sounds at speed	`currentTime` isn't reset before `play()` (section 8).
MINOR Stale state on restart	`playGame` resets the head's position but not `snakeHead.square`, `apple.square`, `currentApple` or the `tail` object. It mostly corrects itself within a tick, but a single `reset()` that rebuilds *all* state is safer.

In the teacher's

BUG	WHAT HAPPENS / WHY
BUG Apple off-screen	`getRandomNumber(max)` can return `max`, so the apple can land one cell past the right or bottom edge (section 7).
BUG Snake can fold over itself	Two quick key presses get around `opposites`, and the collision loop starting at `i = 3` doesn't notice (section 6).

MINOR Apple drawn over the head	`gameLoop` draws the snake, *then* the apple, then checks for eating, so on the eating frame the apple covers the head.
MINOR Resize	The canvas can shrink with the snake or apple outside it, which leads to a sudden wall death or an apple that can't be reached.
MINOR Apple image not awaited	Only `snakeHead.onload` gates the start of the game.

11. Features only one version has

Only in yours

- Direction-aware head, tail and corner sprites
- Open-mouth head when on the apple
- Background music, mute/unmute
- Crash → game-over sound chaining
- New Game button with a full restart
- DOM score display and game-over card
- Title image and styled overlay UI (CSS)
- Grid-based O(1) occupancy lookup

Only in the teacher's

- Reverse-direction guard via a lookup table
- Allowing reverse when the snake is a single piece
- Waiting for an image to load before starting the loop
- Audio rewind (`currentTime = 0`)
- Snake starts off-screen (`x: -SNAKE_SIZE`) so the first move brings it onto the board
- Class fields, getter, static method, `.at()`, `.some()`

12. What to take from each

Keep from yours: the grid indexing idea, sampling apples from empty cells, the sprite selection logic, and the game-flow UI (start, restart, game over, mute).

Adopt from the teacher's:

1. A single `segments` array with `x, y` only, and a `head` getter.
2. Pop-and-unshift movement, and grow by pushing a copy of the tail.
3. Compute the candidate position → validate it → commit it, all in one place.
4. Recursive `setTimeout` plus a `gameOver` flag.
5. Preload images once, then clear and redraw everything each frame.
6. Table lookups (`opposites`) instead of repeated branches.
7. `audio.currentTime = 0` before replaying.

Add to both: an input buffer (one direction change per tick), safe random ranges, resize handling (pause the game, or keep the board a fixed size), a win condition when the board is full, and a clear split between `update()` and `render()`.